

EvoCracker

A Browser Roguelike with Evolutionary AI

Phase 6 — Final Technical Report

Contributors

Ahmad Mursaleen Shajee Haroon Abdur Rehman Shah

Team: AVM4D | May 3, 2026

EvoCracker is an AI-first dungeon game in which enemies evolve to counter player behaviour across repeated runs. The project combines a PixiJS rendering runtime, React overlays, eight custom search algorithms, behaviour trees, and a genetic adaptation loop. This report documents the final architecture, key implementation decisions, deployment completion, measured outcomes, and thirteen severe engineering incidents that shaped the release.

Live deployment: <https://evocracker.vercel.app/>

Source repository: <https://github.com/AVM4D/EvoCracker>

Contents

1	Introduction	4
2	System Architecture Overview	4
2.1	Executive Technical Summary	4
2.2	Subsystem Responsibilities	5
3	Architecture Diagrams	5
3.1	Runtime Frame Pipeline	5
3.2	AI Decision Architecture	6
3.3	Request-Path to Movement Architecture	6
3.4	State and Telemetry Architecture	7
3.5	Deployment Architecture	7
3.6	Screen and Progression Architecture	7
3.7	Evidence and Observability Architecture	8
4	Implementation Summary by Phase	8
5	Game Brain: Technical Deep Dive	8
5.1	Search Algorithms	8
5.2	Behaviour Trees	8
5.3	Genetic Algorithm	9
5.4	Behaviour-Tree Decision Contract	9
5.5	Algorithm Quality Controls	9
5.6	Key Architectural Decisions	9
5.7	Non-Functional Requirements Coverage	10
6	Findings and Lessons Learned	10
6.1	What Worked Well	10
6.2	What Was Difficult	10
6.3	Critical Incident Register	11
7	Coding Nightmares: Severe Incident Post-Mortems	11
7.1	Nightmare 1 — Asset Transparency Corruption	11
7.2	Nightmare 2 — IDS, DLS, and Hill Climbing Instability	11
7.3	Nightmare 3 — Alert-State Freeze Loops	11
7.4	Nightmare 4 — Telemetry Drift under High-Frequency Updates	11
7.5	Nightmare 5 — Map and Algorithm Mapping Mismatches	12
7.6	Nightmare 6 — First-Frame Delta Spikes	12
7.7	Nightmare 7 — Initialisation Race during Screen Transitions	12
7.8	Nightmare 8 — Stale Path Targets under Rapid Player Movement	12
7.9	Nightmare 9 — Ally Behaviour Deadlock under Shared Enemy Semantics	12
7.10	Nightmare 10 — Tilemap Mixed-Source Inconsistencies	12
7.11	Nightmare 11 — Analytics Card Duplication and Mislabelling	12
7.12	Nightmare 12 — Floor-Intro Metadata in Incorrect Contexts	12
7.13	Nightmare 13 — Asset Memory Pressure during Long Sessions	13
8	Verification and Quality Assurance	13
8.1	Functional Validation	13

8.2	Release Validation Checklist	13
9	Risk, Limitations, and Future Work	13
9.1	Current Limitations	13
9.2	Technical Risks	13
9.3	Recommended Future Work	13
10	Hosting and Release Readiness	13
10.1	Hosting Objective	14
10.2	Production Endpoints	14
10.3	Deployment Checklist	14
10.4	Operational Release Criteria	14
11	Conclusion	14

1 Introduction

EvoCracker began as an attempt to build a dungeon game in which enemy AI is not scripted once and left static. The core idea was to let enemies adapt between rounds using telemetry derived from real player sessions. Realising that goal required three systems to coexist: a solid real-time game runtime, an observable AI stack, and a reproducible evolution loop.

The production deployment is available at <https://evocracker.vercel.app/>; the full source lives at <https://github.com/AVM4D/EvoCracker>. Implementation is documented across the phase documents docs/PHASE_1...PHASE_5 and finalised here with architecture detail, failure post-mortems, and release notes.

2 System Architecture Overview

EvoCracker follows a layered design where rendering, AI, gameplay systems, and analytics are decoupled but connected through clear contracts.

2.1 Executive Technical Summary

- **Runtime model.** Single-threaded render/game loop with async pathfinding requests and deterministic state snapshots.
- **AI model.** Three-layer stack: Behaviour Tree orchestration, search-based navigation, and GA adaptation.
- **Data model.** Centralised Zustand state with persisted run history and iteration proofs.
- **Ops model.** Static frontend hosting with Supabase-backed leaderboard and telemetry persistence.

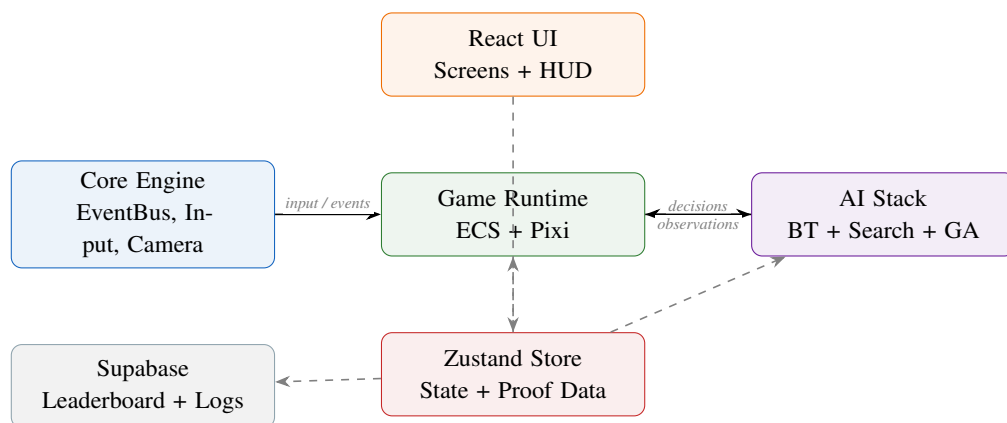


Figure 1. High-level architecture and subsystem interaction.

2.2 Subsystem Responsibilities

Layer	Responsibility
<code>src/core</code>	Runtime primitives: input polling, camera interpolation, event transport, sprite/asset loading.
<code>src/game</code>	World generation, entities, systems, combat, vision, map rendering, and item logic.
<code>src/ai</code>	Search algorithms, behaviour trees, genome model, fitness scoring, and evolution loop.
<code>src/store</code>	Global state: routing, player state, evolution records, analytics snapshots, and persistence.
<code>src/ui</code>	Main menu, map/character selection, gameplay screen, Algorithm Lab, and analytics dashboard.
<code>src/services</code>	Supabase integration for leaderboard and evolution telemetry persistence.

3 Architecture Diagrams

3.1 Runtime Frame Pipeline

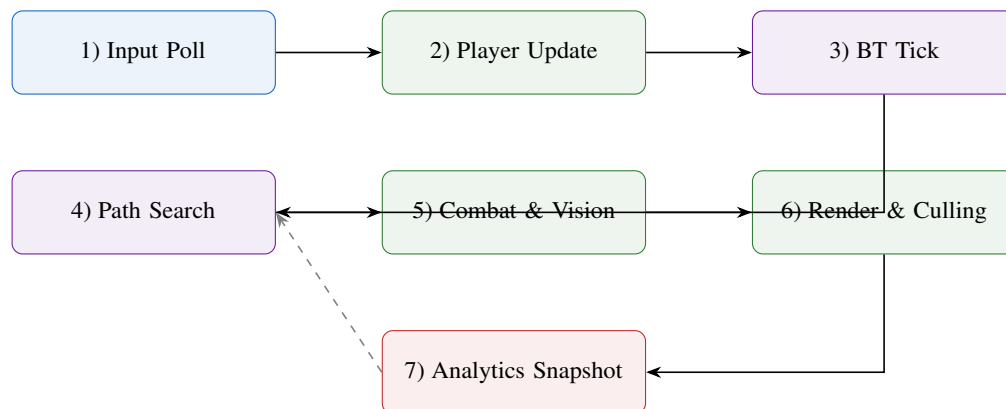


Figure 2. Single-frame execution path with AI feedback loop.

3.2 AI Decision Architecture

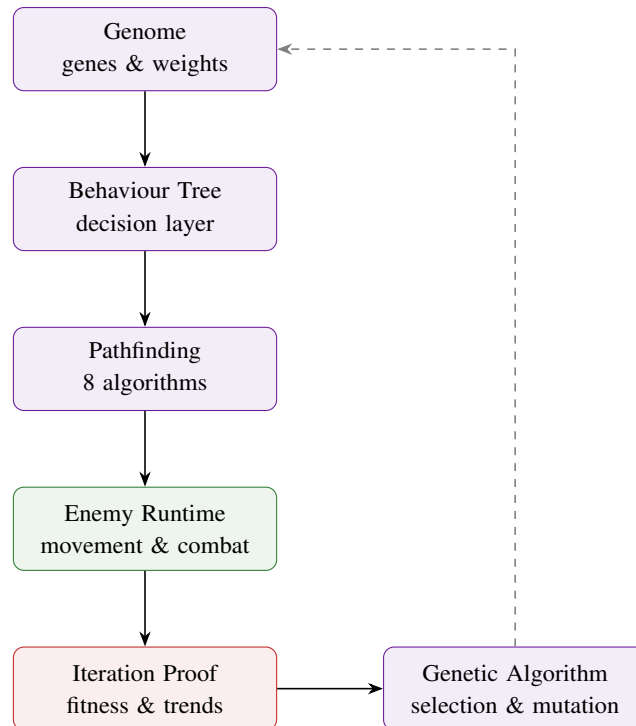


Figure 3. Closed-loop adaptation cycle (co-evolution).

3.3 Request-Path to Movement Architecture

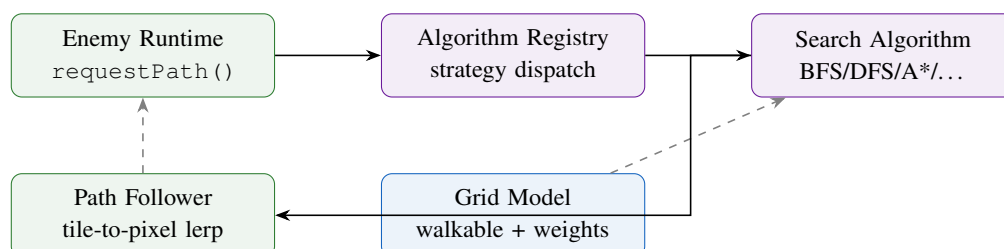


Figure 4. Path request and movement update pipeline.

3.4 State and Telemetry Architecture

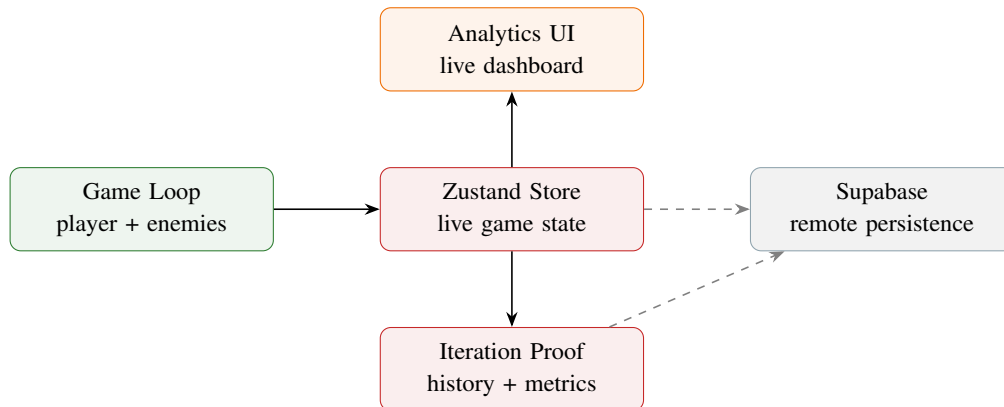


Figure 5. State flow from runtime telemetry to visual proofs and persistence.

3.5 Deployment Architecture

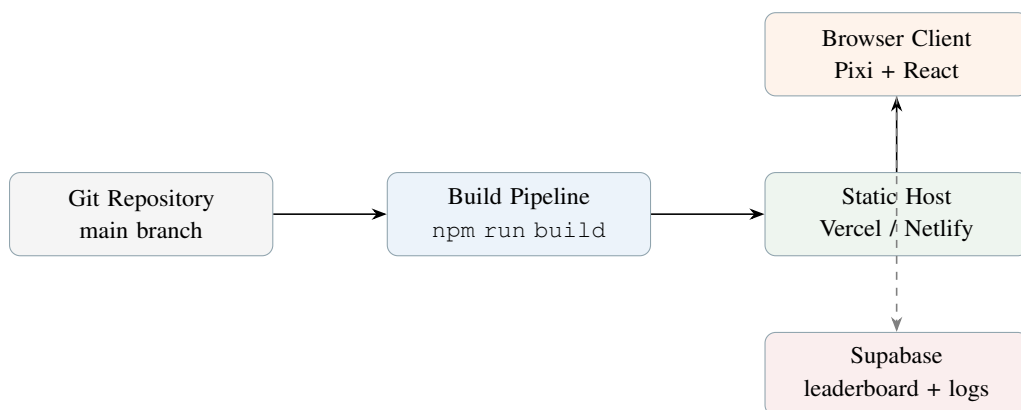


Figure 6. Hosting topology for final delivery.

3.6 Screen and Progression Architecture

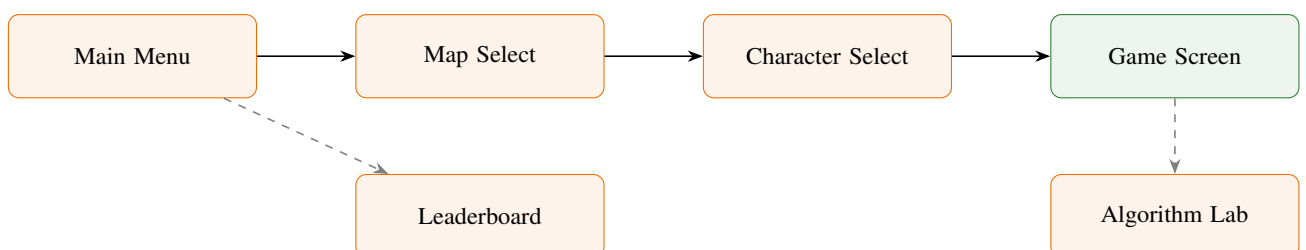


Figure 7. UI flow architecture and major navigation transitions.

3.7 Evidence and Observability Architecture

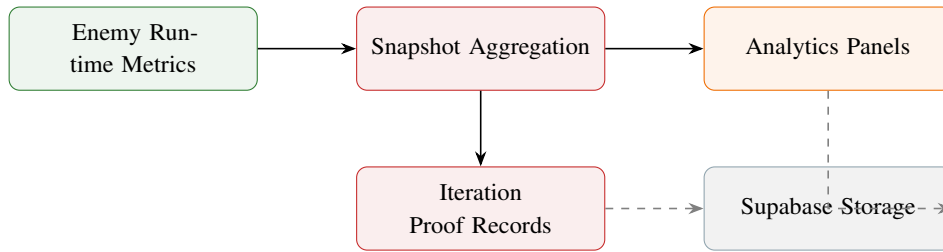


Figure 8. Observability architecture used for proof and comparative analysis.

4 Implementation Summary by Phase

Phase	Deliverables
Phase 1	Core architecture, state-space definition, entity-system baseline, and foundational runtime controls.
Phase 2	Calibration setup with telemetry capture (keystrokes, movement coordinates, zone dwell time).
Phase 3	Search algorithms, behaviour tree runtime integration, and genetic algorithm pipeline.
Phase 4	Live proof and comparative analysis through analytics metrics and evolution charts.
Phase 5	Playable game flow completion with progression loop and dashboard integration.
Phase 6	Final report, architecture documentation, deployment readiness, and lessons learned.

5 Game Brain: Technical Deep Dive

5.1 Search Algorithms

Eight algorithms are implemented from scratch and exposed through a strategy registry:

- **Uninformed:** BFS, DFS, IDS, DLS.
- **Cost- and heuristic-driven:** UCS, A*, Greedy Best-First Search.
- **Local optimisation:** Hill Climbing.

Every run returns path, nodes expanded, visited nodes, and compute time to power real-time comparative analysis.

5.2 Behaviour Trees

Each enemy carries a behaviour tree with tactical branches for patrol, investigate, chase, and retreat. Blackboard state includes player visibility, last known coordinates, health state, and timers.

5.3 Genetic Algorithm

At iteration boundaries, genomes are evaluated and evolved according to the fitness function:

$$f(g) = 0.3v + 2.0d + 1.5e + 0.2s + 0.4c - 1.0k + 2.5p \quad (1)$$

where v = visible time, d = damage dealt, e = detections, s = survival, c = coverage, k = stuck penalty, and p = cooperative bonus. Selection, crossover, and mutation produce the next population and write proof records.

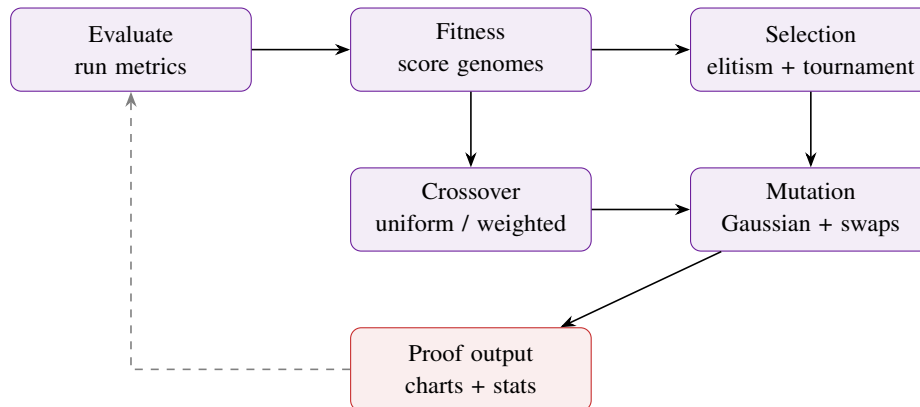


Figure 9. Evolution pipeline executed between game iterations.

5.4 Behaviour-Tree Decision Contract

To keep enemy behaviour debuggable and comparable, each behaviour cycle follows a five-step contract:

1. **Sense.** Vision and proximity checks update blackboard fields.
2. **Select.** The BT chooses a tactical branch (patrol / search / chase / flee).
3. **Plan.** The active branch requests a path using the assigned algorithm and current constraints.
4. **Act.** The path follower advances, combat checks run, and runtime metrics update.
5. **Record.** A snapshot is emitted for analytics and evolution scoring.

5.5 Algorithm Quality Controls

- Standardised output shape: path, expanded nodes, visited nodes, runtime, success flag.
- Consistent `Grid` interface and tile-weight semantics across all algorithms.
- Dashboard instrumentation sourced from live enemies, not synthetic benchmark harnesses.

5.6 Key Architectural Decisions

- **Centralised store over ad-hoc local state** — keeps runtime telemetry, evolution history, and UI views coherent.
- **Registry-based algorithm dispatch** — preserves algorithm modularity and simplifies comparative instrumentation.
- **Behaviour tree orchestration** — keeps tactical decisions interpretable and debuggable compared to opaque policy logic.
- **Iteration proof as first-class artefact** — progress claims remain auditable and reproducible.

5.7 Non-Functional Requirements Coverage

Requirement	Coverage
Performance	Delta clamps, culling, controlled path-request cadence, and lightweight snapshot updates.
Reliability	Guarded state transitions, cleanup ownership boundaries, and fallback asset loading paths.
Observability	Live runtime snapshots, algorithm metrics, and iteration proof history in dashboard views.
Maintainability	Layered architecture, shared interfaces, normalised mappings, and complete phase documentation.

6 Findings and Lessons Learned

6.1 What Worked Well

- Event-driven subsystem boundaries and centralised state made debugging practical at every layer.
- The Algorithm Lab significantly reduced AI debugging time by making algorithmic behaviour visible in isolation.
- Iteration proof records turned adaptation claims into measurable evidence rather than anecdote.

6.2 What Was Difficult

- Balancing real-time game loops against React overlay reactivity required careful ownership boundaries.
- Preserving algorithm diversity without allowing one strategy to dominate required ongoing fitness-weight tuning.
- Supporting mixed art and map pipelines while keeping the runtime stable proved unexpectedly complex.

6.3 Critical Incident Register

Incident	Production Impact	Containment Strategy
Alert-state oscillation loop	Enemies and allies entered repeated stun cycles and froze during live rounds.	Guarded state transitions; anti-loop conditions.
Algorithm instability under dynamic updates	IDS, DLS, and Hill Climbing produced non-recovering or low-quality paths.	Path-refresh policy redesign and fallback routing.
Asset pipeline corruption	Character sprites rendered with halos and artefacts, reducing readability.	Deterministic background removal and safer loading fallbacks.
Naming divergence across enums and assets	Characters missing from algorithm mapping and analytics views.	Cross-layer naming normalisation and explicit mapping checks.

7 Coding Nightmares: Severe Incident Post-Mortems

7.1 Nightmare 1 — Asset Transparency Corruption

Symptom. Early custom-generated assets repeatedly lost detail or developed colour halos during background removal.

Resolution. Added deterministic background-removal logic to the sprite loading pipeline together with safer fallback behaviour.

7.2 Nightmare 2 — IDS, DLS, and Hill Climbing Instability

Symptom. IDS, DLS, and Hill Climbing initially produced unstable movement under dynamic combat conditions.

Resolution. Improved path-refresh policy, fallback behaviour, and analytics instrumentation for path progress and compute cost.

7.3 Nightmare 3 — Alert-State Freeze Loops

Symptom. Alert/chasing oscillations could repeatedly trigger stun-style transitions, freezing movement in edge cases.

Resolution. Added guarded transition logic and anti-loop conditions on alert-state changes.

7.4 Nightmare 4 — Telemetry Drift under High-Frequency Updates

Symptom. During high-action scenes the runtime state and overlay snapshots drifted temporarily, producing misleading analytics cards.

Resolution. Enforced tighter update-cadence boundaries and clearer ownership of snapshot writes in the store pipeline.

7.5 Nightmare 5 — Map and Algorithm Mapping Mismatches

Symptom. Character and enemy names diverged in enums and sprite definitions, breaking algorithm labels and analytics consistency.

Resolution. Normalised naming across constants, sprite definitions, enemy factories, and analytics mappings.

7.6 Nightmare 6 — First-Frame Delta Spikes

Symptom. Rare frame-time spikes caused sudden movement jumps and unstable physics/path behaviour.

Resolution. Added delta clamps and stabilised movement and path-update cadence.

7.7 Nightmare 7 — Initialisation Race during Screen Transitions

Symptom. Rapid screen transitions could overlap teardown and re-init logic, causing duplicate runtime objects or orphaned listeners.

Resolution. Introduced stricter cleanup ownership and abort guards before re-initialisation.

7.8 Nightmare 8 — Stale Path Targets under Rapid Player Movement

Symptom. Enemies occasionally pursued stale targets when player movement changed direction rapidly, producing visibly delayed reactions.

Resolution. Reworked path-target invalidation and request gating to prefer current tactical context.

7.9 Nightmare 9 — Ally Behaviour Deadlock under Shared Enemy Semantics

Symptom. Allies reused enemy runtime logic and inherited alert semantics incompatible with ally-follow behaviour, causing lock states.

Resolution. Separated ally alert handling and disabled hostile alert transitions in ally steering flow.

7.10 Nightmare 10 — Tilemap Mixed-Source Inconsistencies

Symptom. Legacy tileset assumptions and Tiled JSON data disagreed on GID/animation semantics, producing rendering anomalies.

Resolution. Standardised GID normalisation and added explicit tileset-resolution fallback paths.

7.11 Nightmare 11 — Analytics Card Duplication and Mislabelling

Symptom. Genome and algorithm views showed repeated cards due to incomplete mapping keys and name collisions.

Resolution. Added stricter mapping keys and normalised label sources from runtime snapshot fields.

7.12 Nightmare 12 — Floor-Intro Metadata in Incorrect Contexts

Symptom. Floor name overlays appeared at match start in contexts where they were inaccurate or noisy, reducing UX quality.

Resolution. Removed incorrect floor-intro overlays and restricted progression messaging to valid events.

7.13 Nightmare 13 — Asset Memory Pressure during Long Sessions

Symptom. Long sessions with multiple screen transitions increased texture and object retention risk, degrading performance on lower-end devices.

Resolution. Tightened cleanup strategy for temporary graphics, animated sprites, and transition artefacts.

8 Verification and Quality Assurance

8.1 Functional Validation

- Calibration runs correctly record keystrokes, tile movement, and zone dwell metrics.
- All eight search algorithms execute through the same registry and return comparable metrics.
- Iteration completion writes proof records (strength deltas, algorithm distributions, fitness trends).
- The analytics dashboard updates from live enemy snapshots during gameplay.

8.2 Release Validation Checklist

- No known critical runtime blockers in the main gameplay loop.
- Map select, character select, and game-start transitions are stable.
- Allies and enemies move and attack reliably without lock-state regressions.
- Report and phase documentation are aligned with evaluator requirements.

9 Risk, Limitations, and Future Work

9.1 Current Limitations

- Browser performance varies substantially by GPU/driver path and can affect rendering smoothness.
- Evolution quality depends on run diversity; short repetitive runs can bias adaptation.
- Static-host operations rely on external environment correctness for Supabase connectivity.

9.2 Technical Risks

- Adding new enemy archetypes can reintroduce mapping drift if naming contracts are not enforced.
- Algorithm-level tweaks can unintentionally bias comparative analytics if the output schema changes.
- Additional visual assets may increase memory pressure and startup time.

9.3 Recommended Future Work

- A formal benchmark harness for deterministic algorithm timing across map classes.
- Stronger schema validation between enemy types, character definitions, and analytics labels.
- A CI pipeline step to compile this $\text{\LaTeX}$ report and validate that no broken references remain.

10 Hosting and Release Readiness

10.1 Hosting Objective

Phase 6 emphasised shipping and documentation completion. EvoCracker is static-host ready: the Vite build output is self-contained and Supabase credentials are externalised via environment variables, so no server-side code is required.

10.2 Production Endpoints

- **Hosted application:** <https://evocracker.vercel.app/>
- **Source repository:** <https://github.com/AVM4D/EvoCracker>

10.3 Deployment Checklist

Build: `npm run build` completes without errors.

Output: Production bundle generated in `dist/`.

Routing: Host configured for SPA fallback to `index.html`.

Environment: Supabase URL and anon key set on the host.

Smoke test: Gameplay loop, analytics, and leaderboard API calls verified.

10.4 Operational Release Criteria

Criterion	Status	Evidence
Public hosting available	Complete	Vercel deployment reachable via production URL.
Repository traceability	Complete	Public GitHub repository with docs and source tree.
Dashboard visibility	Complete	In-game analytics panel active and data-backed.
Supabase integration	Complete	Leaderboard and run-logging paths fully integrated.

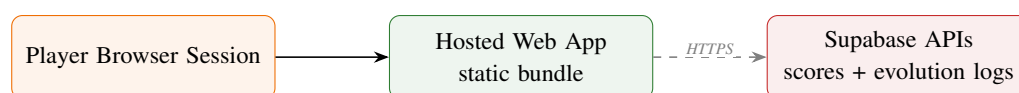


Figure 10. Production runtime relationship after deployment.

11 Conclusion

EvoCracker delivered both a playable product and a serious AI engineering artefact. Its layered architecture scales gracefully, the adaptation loop is measurable and auditable, and the final system is deployment-ready on standard static hosting. This report captures not only what was built, but also the practical engineering journey — thirteen severe incidents and the hard-won stability they produced — that made it ship.

Project Links

Live application: <https://evocracker.vercel.app/>

Source code: <https://github.com/AVM4D/EvoCracker>

Contributors: Ahmad Mursaleen | Shajee Haroon | Abdur Rehman Shah